

Projektheft

Berührungslose Smart-Home-Steuerung

mit MediaPipe · MQTT · ESP32 · Wokwi · Dashboard

Projektzeitraum: 16. Juni 2026 – 07. Juli 2026

Abschlusspräsentation: 07. Juli 2026

Teams: 7 × Zweierteam | 1 × Dreierteam (17 Schülerinnen und Schüler)

Dreierteam-Erweiterung: Docker + Node-RED

1. Kundenauftrag

Ein Unternehmen möchte einen Prototyp für ein modernes Smart-Home-System entwickeln. Die Bedienung soll berührungslos über Handgesten erfolgen. Sensoren und Aktoren werden über einen ESP32 in Wokwi simuliert, die Kommunikation läuft über MQTT. Zusätzlich soll ein Dashboard entwickelt werden, das die Wohnung grafisch darstellt und alle Zustände in Echtzeit visualisiert.

Das Projekt zeigt, wie KI, IoT und moderne Softwareentwicklung zusammenarbeiten. Ziel ist eine möglichst realistische Smart-Home-Anwendung.

2. Projektziel

Entwickeln Sie ein Smart-Home-System mit folgenden Funktionen:

- Gestensteuerung mit MediaPipe
- MQTT-Kommunikation über einen öffentlichen Broker
- Sensorik und Aktorik mit ESP32 in Wokwi
- Grafische Wohnungsvisualisierung (Dashboard)
- Gemeinsame Entwicklung mit GitHub

Das System muss als Gesamtlösung funktionieren und in einer Abschlusspräsentation am 07.07.2026 demonstriert werden.

3. Technische Anforderungen

MediaPipe

- Mindestens zwei Gesten erkennen
- Jede Geste erzeugt eine MQTT-Nachricht

MQTT

- Öffentlichen Broker verwenden (z. B. broker.hivemq.com)
- Sinnvolle und dokumentierte Topic-Struktur

Wokwi / ESP32

- LED, Taster, DHT22, PIR-Sensor und Servo integrieren
- Als MQTT-Subscriber agieren und auf Nachrichten reagieren

Dashboard

- Frei wählbare Technologie
- MQTT-Daten in Echtzeit anzeigen und Zustände visualisieren
- Optional: Geräte über das Dashboard direkt steuern

GitHub

- Gemeinsames Repository mit aussagekräftigem README
- Regelmäßige, nachvollziehbare Commits (mind. ein Commit pro Projekttag)

4. Systemarchitektur

Die Komponenten kommunizieren ausschließlich über MQTT:



Das Dashboard kann zusätzlich MQTT-Nachrichten senden, um Geräte zu steuern. Die Schnittstelle ist immer MQTT.

5. Dashboard-Anforderungen

Das Dashboard ist ein zentraler Bestandteil des Projekts. Die Wohnung wird grafisch mit mindestens folgenden Elementen dargestellt:

- Lichtzustände sichtbar (ein/aus)
- Rollo geöffnet/geschlossen
- Temperatur anzeigen (DHT22-Wert)
- Bewegungsmelder (PIR) visualisieren
- MQTT-Daten in Echtzeit aktualisieren
- Übersichtliche und bedienbare Oberfläche

Beispiel: Wird das Wohnzimmerlicht eingeschaltet, ändert sich die Darstellung sofort. Wird das Rollo bewegt, reagiert die Visualisierung unmittelbar.

6. MQTT-Topics (Vorschlag)

Eine eigene Topic-Struktur ist erlaubt, muss aber logisch aufgebaut und dokumentiert werden.

- home/livingroom/light
- home/livingroom/blinds
- home/kitchen/temperature
- home/bedroom/pir
- home/button
- home/gesture

7. Arbeitsteilung

Vorgeschlagene Aufgabenteilung (Zweierteam). Anpassungen nach Absprache möglich.

Schüler A – Gestenerkennung & Publisher

- MediaPipe-Integration
- Gestenerkennung (mind. 2 Gesten)
- MQTT-Publisher

Schüler B – Hardware & Subscriber

- ESP32 in Wokwi
- Sensoren (DHT22, PIR) und Aktoren (LED, Servo)
- MQTT-Subscriber

Gemeinsam

- Dashboard
- Tests und Fehlerbehebung
- Dokumentation
- Abschlusspräsentation

8. Zeitplan und Meilensteine

Das Projekt läuft in 4 Kalenderwochen (KW 25–28). Folgende Termine und Meilensteine sind verbindlich:

KW	Datum	Meilenstein	Aufgaben / Ziele
KW 25	16.06.2026	Projektstart	Planung, GitHub-Setup, MQTT-Konzept erstellen
KW 25	18.06.2026	Prototyp	Wokwi-Prototyp aufbauen, MQTT-Verbindung testen
KW 26	23.06.2026	Kern-Entwicklung	Gestenerkennung, Sensoren und Aktoren integrieren
KW 26	25.06.2026	Integration	MQTT-Kommunikation, erste End-to-End-Tests
KW 27	30.06.2026	Dashboard	Dashboard entwickeln, Gesamtsystem verbinden
KW 27	02.07.2026	Feinschliff	Fehler beheben, Dokumentation fertigstellen
KW 28	07.07.2026	Abgabe & Präsentation	Praesentationsvorführung und Abgabe aller Artefakte

9. Dokumentation

Jeder Schüler führt ein eigenes Tagebuch (digitales oder schriftliches Dokument). Für jeden Projekttag sind festzuhalten:

- Datum
- Erledigte Aufgaben
- Aufgetretene Probleme
- Gefundene Lösungen

Beispiel-Eintrag:

Datum	18.06.2026
Aufgabe	MQTT mit Wokwi verbunden
Problem	Verbindung zum Broker fehlgeschlagen
Lösung	Portfehler korrigiert (1883 statt 1884)

10. Erweiterung für das Dreierteam

Das Dreierteam (1 Team in dieser Klasse) bearbeitet zusätzlich zur Standardaufgabe eine technische Erweiterung des Systems mit Docker und Node-RED.

Aufgaben des 3. Teammitglieds

- Node-RED in Docker betreiben und konfigurieren
- MQTT-Daten in Node-RED empfangen und verarbeiten
- Automatisierungsregeln entwickeln (z. B. Licht bei Bewegung automatisch ein)
- Daten historisieren (z. B. Temperaturverlauf speichern)
- Zusätzliches Node-RED-Dashboard erstellen

Abgrenzung

Die Erweiterung ist unabhängig vom Hauptprojekt umsetzbar. Die einzige Schnittstelle zum Hauptprojekt ist MQTT – Docker und Node-RED abonnieren dieselben Topics.

Bewertungshinweis

Das Dreierteam wird nach denselben Grundkriterien bewertet wie die Zweierteams. Die Docker/Node-RED-Erweiterung fließt positiv in Komplexität und Funktionsfähigkeit ein.

11. Bewertung (100 Punkte)

Funktionsfähigkeit des Gesamtsystems	30	MQTT, Gesten, ESP32, Dashboard
Komplexität und technische Tiefe	15	Erweiterungen, Kreativität, Lösungsansatz
Codequalität	10	Struktur, Lesbarkeit, Kommentare
GitHub-Nutzung	10	Regelmäßige Commits, Nachvollziehbarkeit
Dashboard	10	Visualisierung, Bedienbarkeit
Dokumentation	10	Vollständigkeit, tägliche Einträge
Projektprozess	10	Sichtbare Leistung an jedem Termin
Abschlusspräsentation	5	Vorführung, Inhalt, Auftreten
Gesamt	100	

12. Abgabe und Checkliste

Abgabedatum: 07. Juli 2026 (Abschlusspräsentation)

Abzugeben sind

- GitHub-Repository (vollständig, mit README)
- Quellcode (MediaPipe-Skript, ESP32-Code, Dashboard)
- Wokwi-Projekt (Link)
- Dokumentation (tagesbasiert, je Schüler)
- Präsentation (Datei, mind. 5 Folien)

Vor der Abgabe prüfen

- MQTT-Kommunikation funktioniert durchgängig
- Gesten werden erkannt und lösen die richtigen Aktionen aus
- Sensoren liefern Daten (DHT22, PIR)
- Aktoren reagieren korrekt (LED, Servo)
- Dashboard visualisiert alle Zustände in Echtzeit
- GitHub-Repository ist vollständig und zugänglich
- Dokumentation ist lückenlos (alle Projektstage)